

Introduction to CSS

Making the Web Beautiful — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser** and **VS Code** open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses matter more than being right.

Part 1 — What is CSS?

TRY THIS · BROWSER

Open the HTML file you built in our last class.

Q1. Look at the page you just opened. What does it look like? What do you think is missing?

Part 2 — Inline CSS

TRY THIS · VS CODE

Create `inline.html` with the usual boilerplate. In the `<body>`, add an `<h1>` with your name — but this time, add `style="color: blue;"` inside the tag, like this: `<h1 style="color: blue;">Your Name</h1>`. Open it in Chrome and see what happens. Then try changing `blue` to any other color you like.

Q2. What happened when you opened the page? Based on what you wrote, what do you think `style="color: blue;"` is telling the browser to do?

TRY THIS · VS CODE

In the same `inline.html`, add `background-color: yellow;` right next to `color` in your `<h1>`'s `style` attribute — so it looks something like `style="color: blue; background-color: yellow;"`. Refresh the page in Chrome.

Q3. What changed now? What do you think `background-color` controls, compared to `color`?

Part 3 — Ways of Writing a Color

TRY THIS · VS CODE

In your `inline.html`, change your `<h1>`'s color to `red`. Refresh. Now change it to `#ff0000`. Refresh. Now change it to `rgb(255, 0, 0)`. Refresh.

Q4. What did you notice across these three refreshes? What do you think `#ff0000` and `rgb(255, 0, 0)` mean?

TRY THIS · VS CODE

Now try the same three formats on `background-color` instead — `red`, then `#ff0000`, then `rgb(255, 0, 0)`. Refresh after each.

Q5. Did `background-color` behave the same way `color` did with these three formats? What does that tell you about where keyword, hex, and rgb values can be used?

Part 4 — A Problem with Inline CSS

TRY THIS · VS CODE

In `inline.html`, add 5 more `<h1>` tags below your name — just repeat your name in each one. Give each `<h1>` its own `style="color: red;"`, written separately on every tag. Now go back and change all 5, one by one, to `color: blue;`.

Q6. What was that like — changing all 5, one by one? If this page had 50 headings instead of 5, what would that experience be like?

Instead of writing `style="..."` on every tag, CSS lets you write all your styles once, in a `<style>` block inside `<head>`. This is called **internal CSS**.

TRY THIS · VS CODE

Remove the `style="..."` from all 6 of your `<h1>` tags. In the `<head>`, add a `<style>` block. Inside it, write:

```
h1 {  
  color: blue;  
}
```

Refresh the page.

Q7. What happened to all 6 headings this time? How many lines did you have to change to make this happen, compared to before?

That `h1` before the curly braces is called a **selector** — it tells CSS which element(s) to style. Since `h1` is a tag name, this is called a **tag selector**, and it styles every single `<h1>` on the page.

Q8. Based on that, if you wrote `p { color: green; }` instead, what would happen on your page?

Part 5 — Styling One Specific Element

Q9. Say you have 6 `<h1>` tags, all styled the same way via the `h1` tag selector. Now you want to change the color of **just one of them** — not all 6. The tag selector styles every `<h1>`, so that won't work alone. Any guess how CSS might let you target just one specific element?

TRY THIS · VS CODE

Give one of your 6 `<h1>` tags an `id="special"`. In your `<style>` block, add:

```
#special {  
  color: teal;  
}
```

Refresh.

Q10. What happened? How does writing `#special` in your CSS connect back to `id="special"` in your HTML?

Part 6 — Styling a Group of Elements

Q11. Now say you want to style **3 of your 6** headings the same new way — not all 6 (that's the tag selector), and not just 1 (that's what you just did with `id`). Any guess how you'd target a specific group?

TRY THIS · VS CODE

Give 3 of your `<h1>` tags `class="highlight"`. In your `<style>` block, add:

```
.highlight {  
  color: orange;  
}
```

Refresh.

Q12. What happened to those 3? How is `class` different from `id` — could you reuse the same `id` on multiple elements? Take a guess.

Q13. In your own words, what's the difference between an `id` and a `class`? When would you use one over the other?

Part 7 — External CSS

Q14. Your `<style>` block works great for one page. But imagine you have a whole website — 10 different HTML pages — and you want all of them to share the exact same look (same colors, same fonts). With internal CSS, what would you have to do on each of those 10 pages? What problems do you see with that?

TRY THIS · VS CODE

Create a new file called `style.css` in the same folder. Cut everything from inside your `<style>...</style>` block (just the CSS rules, not the tags) and paste it into `style.css`. Delete the now-empty `<style>` block from `inline.html`. In its place, inside `<head>`, add:

```
<link rel="stylesheet" href="style.css">
```

Refresh the page.

Q15. Did the page still look the same after moving everything to `style.css`? What do you think `<link rel="stylesheet" href="style.css">` is doing?

Q16. Now imagine `style.css` is linked from all 10 pages of a website. If you change one color in `style.css` and save, what happens to those 10 pages? How is this different from internal CSS?

Part 8 — Inline vs. Block Elements

TRY THIS · VS CODE

In your file, add a `<div>` with a background color right before your `<h1>`, and another `<div>` with a background color right after it. Then add two `<span>` tags, each with a different background color, side by side inside a paragraph of text. Refresh.

Q17. Look at the colored backgrounds. Did each `<div>` take up its own full line? Did the two `<span>`s sit next to each other on the same line, or did they break onto new lines?

Q18. Try giving one of your `<div>`s a `width: 200px;`. Now try giving one of your `<span>`s the same `width: 200px;`. What happened in each case? Any guess why `width` behaved differently?

Part 9 — Wrapping Up

Q19. You've now used all three ways of writing CSS today — inline (`style="..."`), internal (`<style>` in the page), and external (a separate `.css` file linked in). In your own words, when would you reach for each one, and why?
